RET Instruction

- *Return from subroutine*
- Pops stack into the instruction pointer (EIP or IP). Control transfers to the target address.
- Syntax:
  - **RET**
  - **RET *n***
- Optional operand *n* causes *n* bytes to be added to the stack pointer after EIP (or IP) is assigned a value.

Who removes parameters from the stack?

Caller (C) ..... or ..... Called-procedure (STDCALL):

```
push val2
push val1
call AddTwo
add   esp,8
```

```
AddTwo PROC
    push  ebp
    mov   ebp,esp
    mov   eax,[ebp+12]
    add   eax,[ebp+8]

    pop   ebp
    ret   8
```

(Covered later: The MODEL directive specifies calling conventions)

Your turn . . .

- Create a procedure named Difference that subtracts the first argument from the second one. Following is a sample call:

```
push 14                         ; first argument
push 30                         ; second argument
call Difference                 ; EAX = 16
```

```
Difference PROC
    push ebp
    mov  ebp,esp
    mov  eax,[ebp + 8]      ; second argument
    sub  eax,[ebp + 12]     ; first argument
    pop  ebp
    ret  8
Difference ENDP
```

Saving and Restoring Registers

- Push registers on stack just after assigning ESP to EBP
  - local registers are modified inside the procedure

```
MySub PROC
  push  ebp
  mov   ebp,esp
  push  ecx           ; save local registers
  push  edx
```

Stack Affected by USES Operator

```
MySub1 PROC USES ecx edx
   ret
MySub1 ENDP
```

- USES operator generates code to save and restore registers:

```
MySub1 PROC
   push  ecx
   push  edx

   pop   edx
   pop   ecx
ret
```

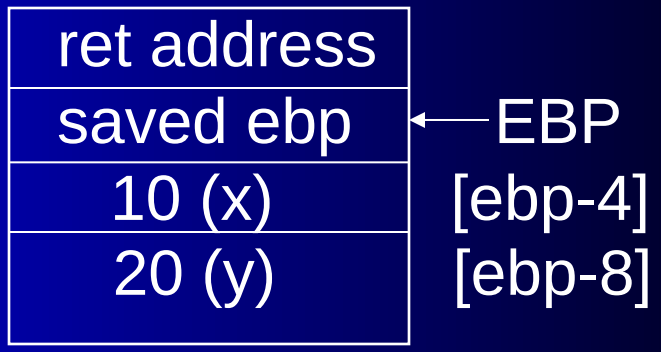
Local Variables

- Only statements within subroutine can view or modify local variables
- Storage used by local variables is released when subroutine ends
- local variable name can have the same name as a local variable in another function without creating a name clash

Creating LOCAL Variables

Example - create two DWORD local variables:

Say: int x=10, y=20;




```
MySub PROC
        push   ebp
        mov    ebp,esp
        sub    esp,8          ;create 2 DWORD variables

        mov    DWORD PTR [ebp-4],10 ; initialize x=10
        mov    DWORD PTR [ebp-8],20 ; initialize y=20
```

LEA Instruction

- LEA returns offsets of direct and indirect operands
  - OFFSET operator only returns constant offsets
- LEA required when obtaining offsets of stack parameters & local variables
- Example

```
CopyString PROC,
    count:DWORD
    LOCAL temp[20]:BYTE

    mov edi,OFFSET count        ; invalid operand
    mov esi,OFFSET temp         ; invalid operand
    lea edi,count               ; ok
    lea esi,temp                ; ok
```

LEA Example

Suppose you have a Local variable at [ebp-8]

And you need the address of that local variable in ESI

You cannot use this:

```
mov esi, OFFSET [ebp-8]          ; error
```

Use this instead:

```
lea esi,[ebp-8]
```

ENTER Instruction

- ENTER instruction creates stack frame for a called procedure
  - pushes EBP on the stack
  - sets EBP to the base of the stack frame
  - reserves space for local variables
  - Example:

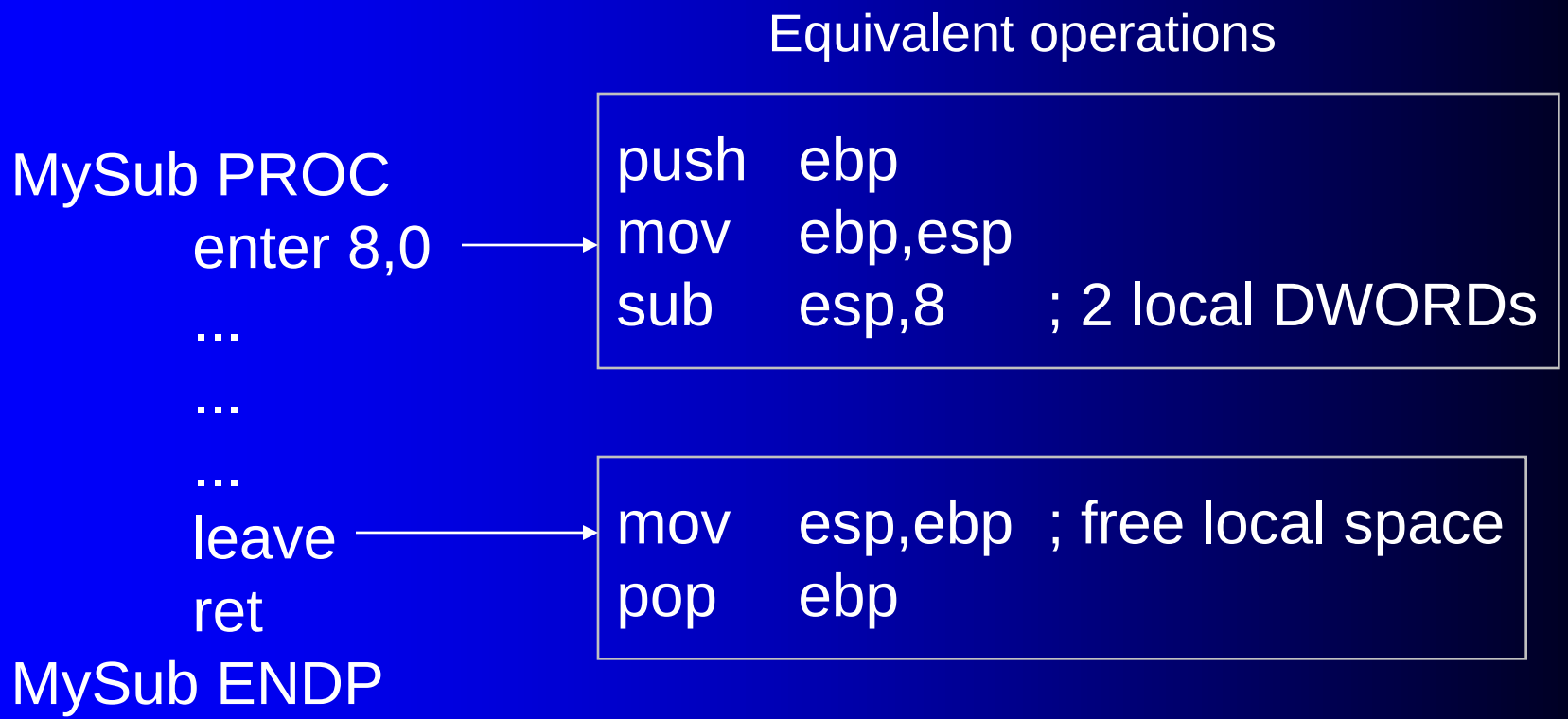
```
MySub PROC
   enter 8,0
```

  - Equivalent to:

```
MySub PROC
   push ebp
   mov ebp,esp
   sub esp,8
```

LEAVE Instruction

Terminates the stack frame for a procedure.



LOCAL Directive

- The LOCAL directive declares a list of local variables
  - immediately follows the PROC directive
  - each variable is assigned a type
- Syntax:

```
LOCAL varlist
```

Example:

```
MySub PROC
    LOCAL var1:BYTE, var2:WORD, var3:SDWORD
```

Using LOCAL

Examples:

```
LOCAL flagVals[20]:BYTE     ; array of bytes

LOCAL pArray:PTR WORD       ; pointer to an array

myProc PROC,                ; procedure
    LOCAL t1:BYTE,          ; local variables
```

LOCAL Example (1 of 2)

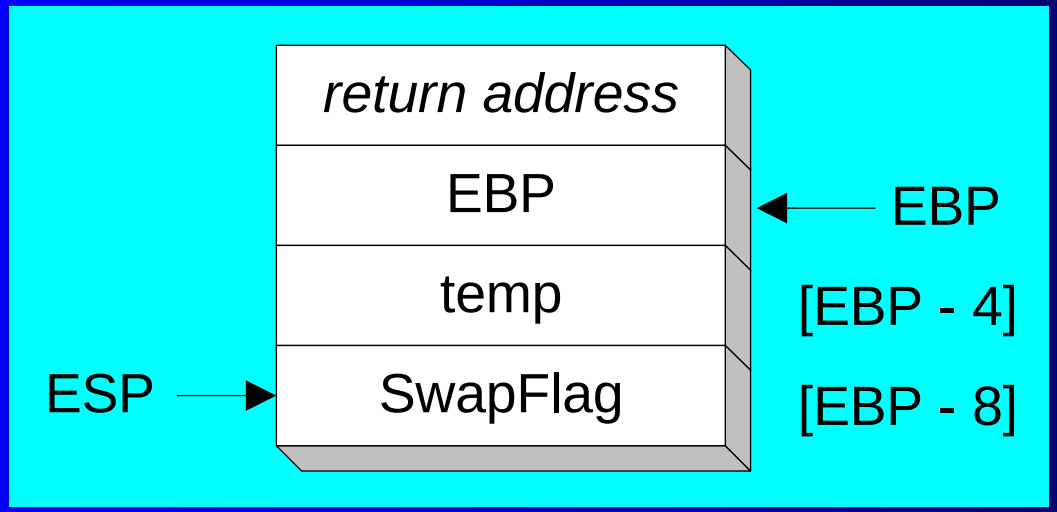
```
BubbleSort PROC
    LOCAL temp:DWORD, SwapFlag:BYTE
    . . .
    ret
BubbleSort ENDP
```

MASM generates the following code:

```
BubbleSort PROC
    push ebp
    mov  ebp,esp
    add  esp,0FFFFFFF8h         ; add -8 to ESP
    . . .
    mov  esp,ebp
    pop  ebp
    ret
BubbleSort ENDP
```

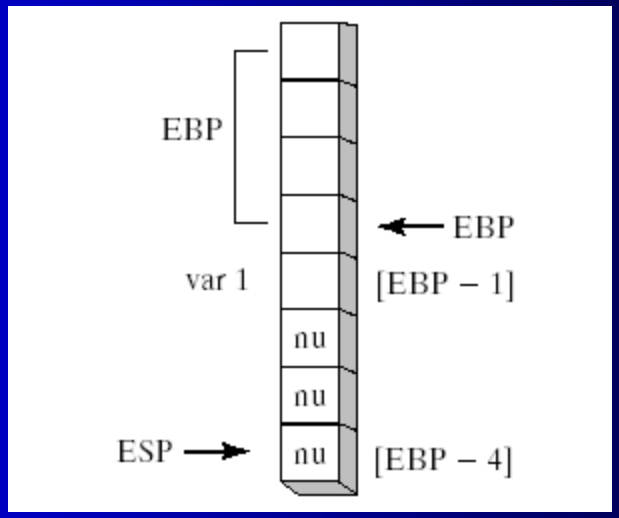
LOCAL Example (2 of 2)

Diagram of the stack frame for the BubbleSort procedure:



Local Byte Variable

```
Example1 PROC
   LOCAL var1:BYTE
   mov al,var1              ; [EBP - 1]
   ret
Example1 ENDP
```



INVOKE Directive

- In 32-bit mode, the INVOKE directive is a powerful replacement for Intel's CALL instruction that lets you pass multiple arguments
- Syntax:

```
INVOKE procedureName [, argumentList]
```

- *ArgumentList* is an optional comma-delimited list of procedure arguments
- Arguments can be:
  - immediate values and integer expressions
  - variable names
  - address and ADDR expressions
  - register names

INVOKE Examples

```
.data
byteVal BYTE 10
wordVal WORD 1000h
.code
    ; direct operands:
    INVOKE Sub1,byteVal,wordVal

    ; address of variable:
    INVOKE Sub2,ADDR byteVal

    ; register name, integer expression:
    INVOKE Sub3,eax,(10 * 20)

    ; address expression (indirect operand):
    INVOKE Sub4,[ebx]
```

PROC Directive (1 of 2)

- The PROC directive declares a procedure with an optional list of named parameters.
- Syntax:

  label PROC paramList
- *paramList* is a list of parameters separated by commas. Each parameter has the following syntax:

  paramName : *type*

type must either be one of the standard ASM types (BYTE, SBYTE, WORD, etc.), or it can be a pointer to one of these types.

PROC Directive (2 of 2)

- Alternate format permits parameter list to be on one or more separate lines:

  label PROC, ← comma required

  paramList

- The parameters can be on the same line . . .

  param-1:type-1, param-2:type-2, . . ., param-n:type-n

- Or they can be on separate lines:

  param-1:type-1,

  param-2:type-2,

  . . .,

  param-n:type-n

AddTwo Procedure (1 of 2)

- The AddTwo procedure receives two integers and returns their sum in EAX.

```
AddTwo PROC,
    val1:DWORD, val2:DWORD

    mov eax,val1
    add eax,val2

    ret
AddTwo ENDP
```

PROC Examples (2 of 3)

FillArray receives a pointer to an array of bytes, a single byte fill value that will be copied to each element of the array, and the size of the array.

```
FillArray PROC,
    pArray:PTR BYTE, fillVal:BYTE
    arraySize:DWORD

    mov ecx,arraySize
    mov esi,pArray
    mov al,fillVal
L1: mov [esi],al
    inc esi
    loop L1
    ret
FillArray ENDP
```